

Travaux Pratiques 5 : Apache Spark - Traitement et Analyse Distribuée de Données Massives

Université / École :

- Université Ibn Tofail, Kénitra

Encadrement pédagogique :

- Pr. Hajar Filali

Rédigé par :

- Mohammed Jebrou
- Hajar Farid

Année universitaire :

- 2025-2026

Contexte, Objectifs et Compétences Visées

1. Introduction

Ce TP a pour objectif de découvrir **Apache Spark**, un framework de traitement distribué largement utilisé dans les environnements Big Data. Face à l'augmentation continue du volume des données, Spark offre des solutions performantes pour le traitement, l'analyse et l'exploitation de données à grande échelle. À travers cette séance, nous explorerons ses principaux concepts et mettrons en pratique ses fonctionnalités essentielles.

2. Objectif

L'objectif de ce TP est de prendre en main l'environnement Apache Spark en réalisant son installation et sa configuration, puis de découvrir les opérations de base de traitement de données à l'aide du Spark Shell. À travers des exercices simples de manipulation de collections et de filtrage de données, ce TP permet de se familiariser avec les concepts fondamentaux de Spark et avec l'utilisation des RDDs pour le traitement distribué des données.

Étape 1 : téléchargement des indpendance

1. Installation de Java (JDK)

- Java est déjà installé

```
mohammed@ROG-Strix:~/Spark
> java -version
openjdk version "1.8.0_482"
OpenJDK Runtime Environment (build 1.8.0_482-8u482-ga~us1-0ubuntu1~24.04-b08)
OpenJDK 64-Bit Server VM (build 25.482-b08, mixed mode)
🔄 Spark >
```

2. Téléchargement du dossier d'Apache Spark

```
mohammed@ROG-Strix:~/Spark
> wget https://downloads.apache.org/spark/spark-4.1.1/spark-4.1.1-bin-hadoop3.tgz
--2026-04-19 14:25:16-- https://downloads.apache.org/spark/spark-4.1.1/spark-4.1.1-bin-hadoop3.tgz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 135.181.214.104, 2a01:4f9:3a:2c57::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 572739279 (546M) [application/x-gzip]
Saving to: 'spark-4.1.1-bin-hadoop3.tgz'

spark-4.1.1-bin-had 100%[=====>] 546.21M  1.21MB/s   in 15m 53s

2026-04-19 14:41:10 (587 KB/s) - 'spark-4.1.1-bin-hadoop3.tgz' saved [572739279/572739279]

> ls
installation  📁 spark-4.1.1-bin-hadoop3.tgz
🔄 Spark >
```

Étape 2: Configuration des Variables d'Environnement

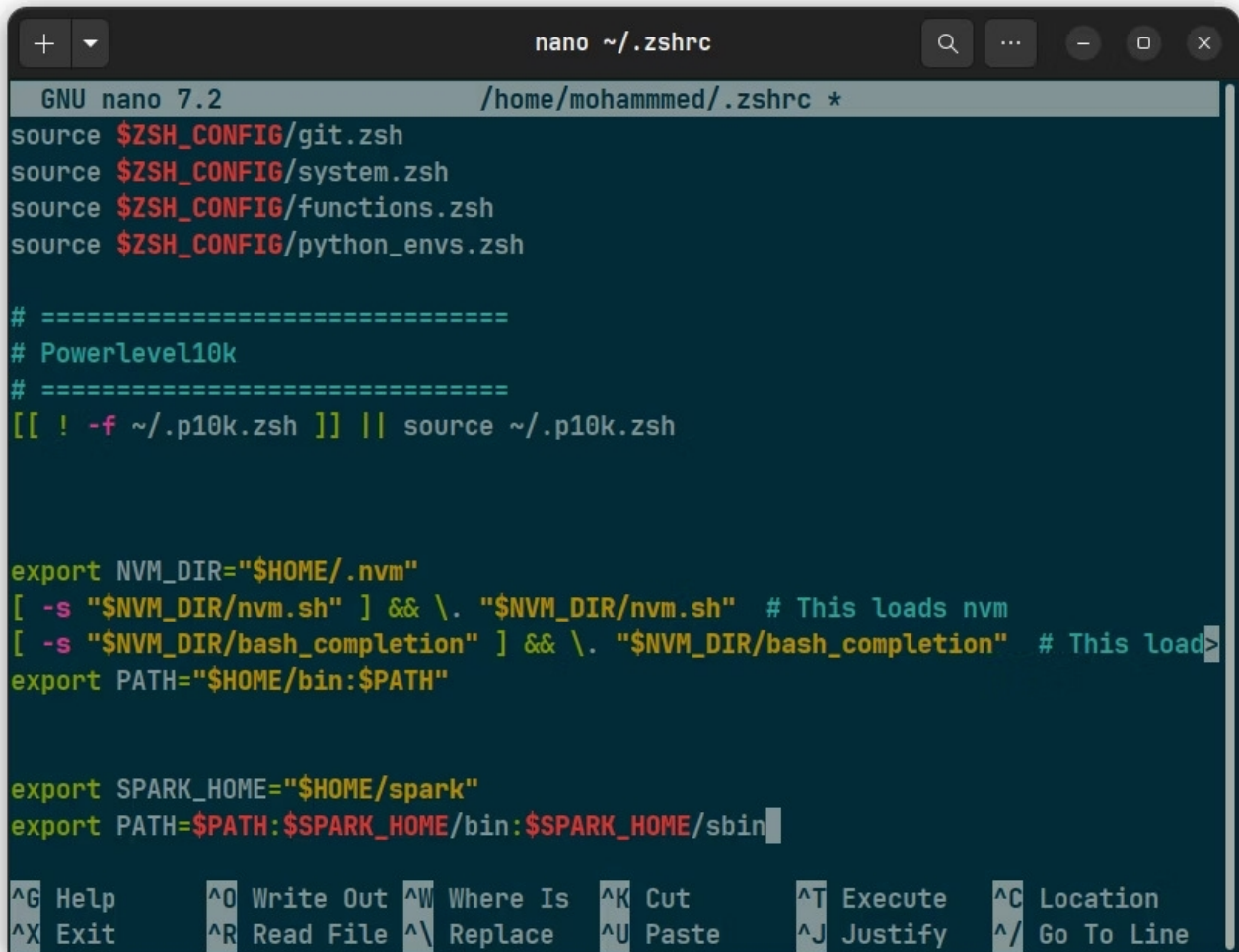
1. Extraction du dossier

```
mohammed@ROG-Strix:~/Spark
> tar -xvf spark-4.1.1-bin-hadoop3.tgz
spark-4.1.1-bin-hadoop3/
spark-4.1.1-bin-hadoop3/conf/
spark-4.1.1-bin-hadoop3/conf/workers.template
spark-4.1.1-bin-hadoop3/conf/fairscheduler.xml.template
spark-4.1.1-bin-hadoop3/conf/spark-defaults.conf.template
spark-4.1.1-bin-hadoop3/conf/log4j2.properties.template
spark-4.1.1-bin-hadoop3/conf/log4j2-json-layout.properties.template
spark-4.1.1-bin-hadoop3/conf/spark-env.sh.template
spark-4.1.1-bin-hadoop3/conf/metrics.properties.template
spark-4.1.1-bin-hadoop3/NOTICE
spark-4.1.1-bin-hadoop3/jars/
spark-4.1.1-bin-hadoop3/jars/datasketches-memory-3.0.2.jar
spark-4.1.1-bin-hadoop3/jars/jline-2.14.6.jar
spark-4.1.1-bin-hadoop3/jars/kubernetes-model-autoscaling-7.4.0.jar
spark-4.1.1-bin-hadoop3/jars/javassist-3.30.2-GA.jar
spark-4.1.1-bin-hadoop3/jars/spark-network-shuffle_2.13-4.1.1.jar
spark-4.1.1-bin-hadoop3/jars/netty-handler-4.2.7.Final.jar
spark-4.1.1-bin-hadoop3/jars/orc-shims-2.2.1.jar
spark-4.1.1-bin-hadoop3/jars/hadoop-client-api-3.4.2.jar
spark-4.1.1-bin-hadoop3/jars/spark-yarn_2.13-4.1.1.jar
spark-4.1.1-bin-hadoop3/jars/avro-1.12.1.jar
spark-4.1.1-bin-hadoop3/jars/spark-kvstore_2.13-4.1.1.jar
spark-4.1.1-bin-hadoop3/jars/jakarta.servlet-api-5.0.0.jar
```

2. Déplacement du Spark vers ~/

```
mohammed@ROG-Strix:~/Spark
> mv spark-4.1.1-bin-hadoop3 ~/spark
🔄 Spark >
```

3. Définition des variables d'environnement

A screenshot of a terminal window with the nano text editor open, editing the file ~/.zshrc. The window title is 'nano ~/.zshrc'. The editor shows the following content: source \$ZSH_CONFIG/git.zsh, source \$ZSH_CONFIG/system.zsh, source \$ZSH_CONFIG/functions.zsh, source \$ZSH_CONFIG/python_envs.zsh, followed by a comment block for Powerlevel10k, then [[! -f ~/.p10k.zsh]] || source ~/.p10k.zsh. Below that, export NVM_DIR="\$HOME/.nvm", followed by two lines for nvm completion: [-s "\$NVM_DIR/nvm.sh"] && \. "\$NVM_DIR/nvm.sh" # This loads nvm and [-s "\$NVM_DIR/bash_completion"] && \. "\$NVM_DIR/bash_completion" # This loads nvm completion. Then export PATH="\$HOME/bin:\$PATH". Further down, export SPARK_HOME="\$HOME/spark" and export PATH=\$PATH:\$SPARK_HOME/bin:\$SPARK_HOME/sbin. The bottom of the screen shows nano shortcuts: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^_ Replace, ^U Paste, ^J Justify, ^/ Go To Line.

```
GNU nano 7.2 /home/mohammed/.zshrc *
source $ZSH_CONFIG/git.zsh
source $ZSH_CONFIG/system.zsh
source $ZSH_CONFIG/functions.zsh
source $ZSH_CONFIG/python_envs.zsh

# =====
# Powerlevel10k
# =====
[[ ! -f ~/.p10k.zsh ]] || source ~/.p10k.zsh

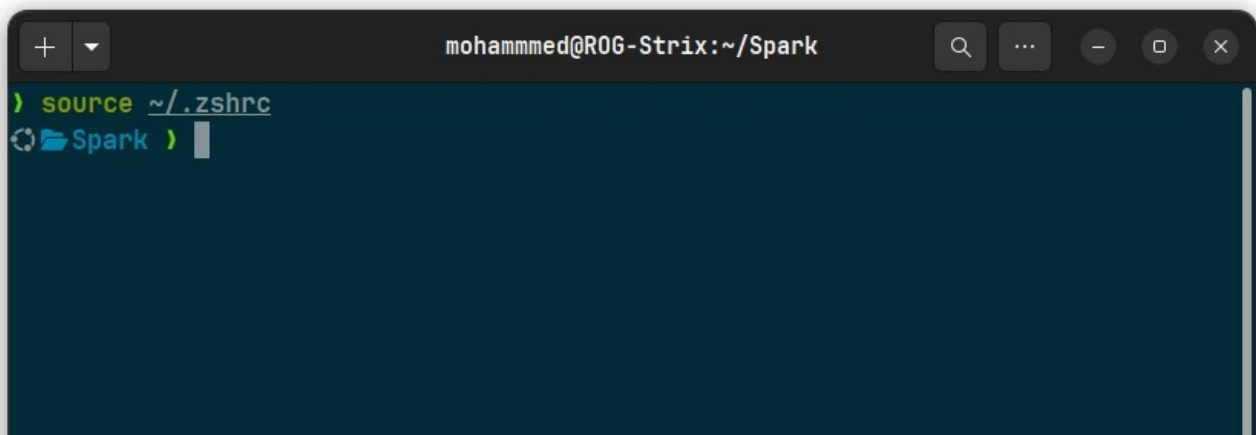
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm completion
export PATH="$HOME/bin:$PATH"

export SPARK_HOME="$HOME/spark"
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin

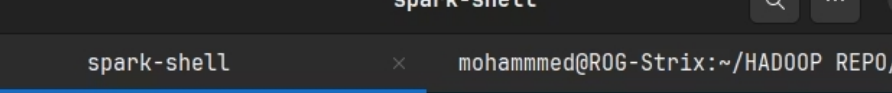
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

4. Application des modifications

Après l'ajout des variables d'environnement de Spark dans le fichier ~/.zshrc, il est nécessaire de recharger la configuration du shell afin que les changements soient pris en compte.

A screenshot of a terminal window with the title 'mohammed@ROG-Strix:~/Spark'. The prompt is ')>'. The user has entered the command 'source ~/.zshrc'. Below the command, there is a small icon of a terminal window and the word 'Spark' followed by a prompt character '>'.

```
mohammed@ROG-Strix:~/Spark
)> source ~/.zshrc
Spark >
```

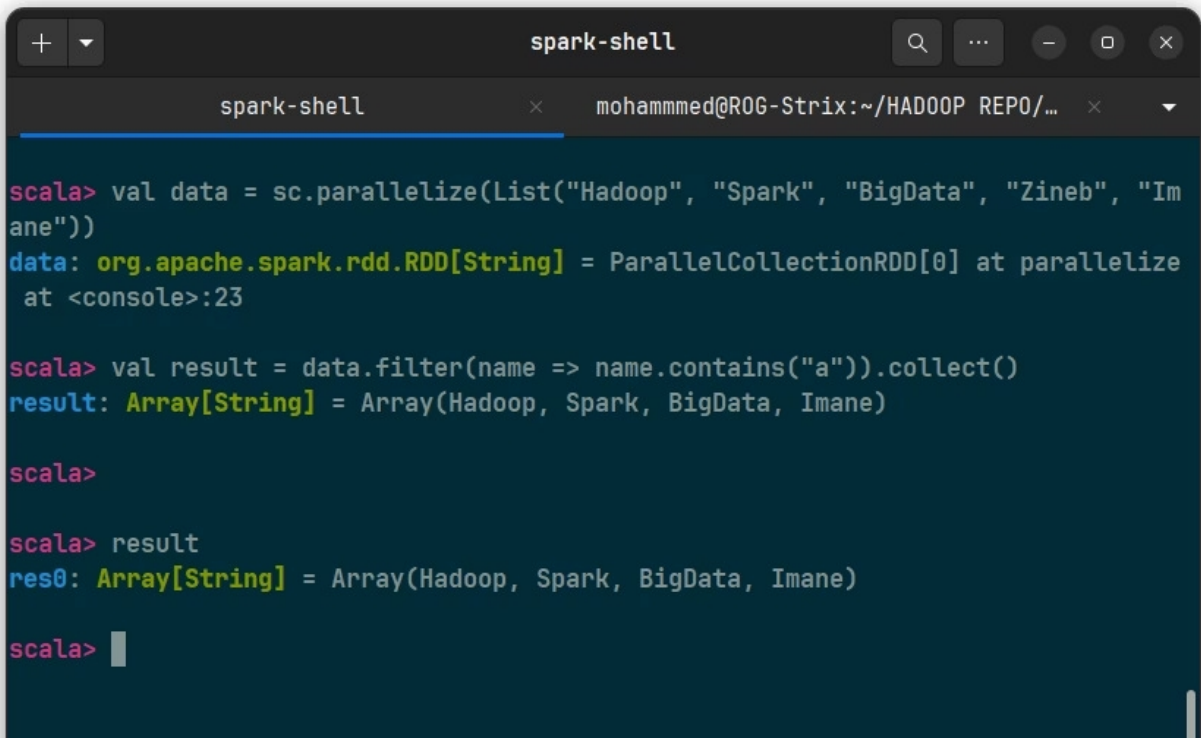
The screenshot shows a terminal window titled "spark-shell". The window has a dark theme. The terminal content shows a Scala prompt "scala>" followed by the command "println(" Spark is working")". The output of the command is "Spark is working". The prompt "scala>" is followed by a cursor.

```
spark-shell
scala> println(" Spark is working")
Spark is working
scala>
```

Exercices Pratiques : Opérations de Traitement de Données avec Apache Spark

1. Extraction des mots contient la lettre "a"

Cette opération utilise la méthode `filter()` afin de sélectionner les éléments contenant le caractère "a".



```
spark-shell
scala> val data = sc.parallelize(List("Hadoop", "Spark", "BigData", "Zineb", "Imane"))
data: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[0] at parallelize at <console>:23

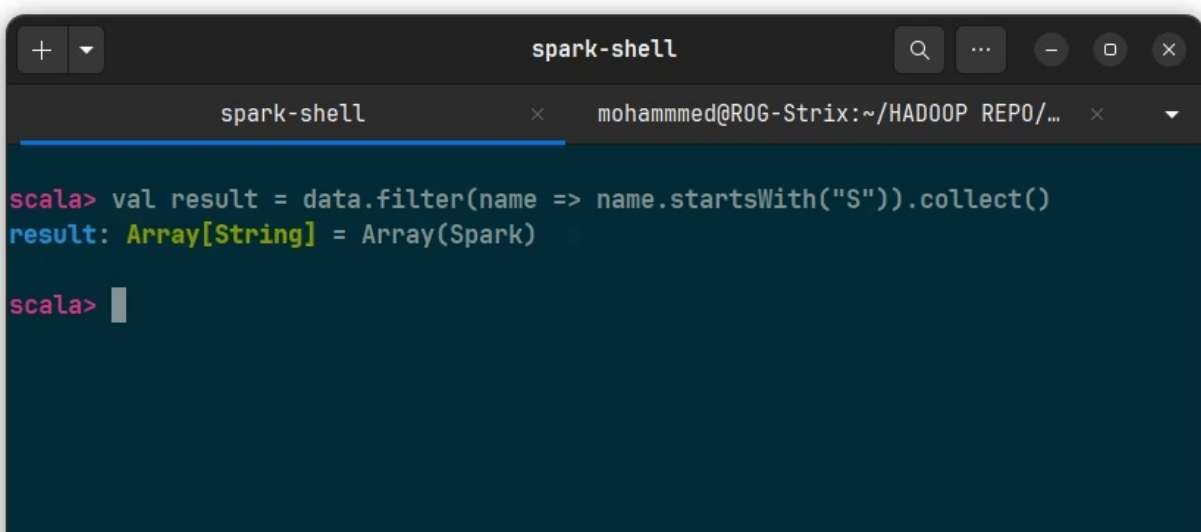
scala> val result = data.filter(name => name.contains("a")).collect()
result: Array[String] = Array(Hadoop, Spark, BigData, Imane)

scala>

scala> result
res0: Array[String] = Array(Hadoop, Spark, BigData, Imane)

scala>
```

2. Extraction des mots commençant par la lettre « S »



```
spark-shell
scala> val result = data.filter(name => name.startsWith("S")).collect()
result: Array[String] = Array(Spark)

scala>
```